visualize_attention

October 6, 2026

[1]:
```
from utils.visualizations import load_images_from_indices, visualize_attention,␣
 ↪visualize_attention_on_images

import io
import json
import math
import os
from datetime import datetime

import matplotlib.cm as cm
import matplotlib.pyplot as plt
import mlflow
import numpy as np
import torch
import torchvision
import torchvision.transforms as transforms
import torchvision.transforms.functional as TF
from PIL import Image
from torch import nn
from torch.nn import functional as F
from model import ViTForClassfication
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[1], line 1
----> 1 from utils.visualizations import load_images_from_indices,␣
 ↪visualize_attention, visualize_attention_on_images
      2
      3 import io
      4 import json

ModuleNotFoundError: No module named 'utils'
```

[2]:
```
def prepare_data(batch_size=4, num_workers=2, train_sample_size=None,␣
 ↪test_sample_size=None):
    train_transform = transforms.Compose(
```

```
        [transforms.ToTensor(),
        transforms.Resize((32, 32)),
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.RandomResizedCrop((32, 32), scale=(0.8, 1.0), ratio=(0.75, 1.
↪3333333333333333), interpolation=2),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

    trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                            download=True,␣
↪transform=train_transform)
    if train_sample_size is not None:
        # Randomly sample a subset of the training set
        indices = torch.randperm(len(trainset))[:train_sample_size]
        trainset = torch.utils.data.Subset(trainset, indices)



    trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                            shuffle=True,␣
↪num_workers=num_workers)

    test_transform = transforms.Compose(
        [transforms.ToTensor(),
        transforms.Resize((32, 32)),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

    testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                        download=True, transform=test_transform)
    if test_sample_size is not None:
        # Randomly sample a subset of the test set
        indices = torch.randperm(len(testset))[:test_sample_size]
        testset = torch.utils.data.Subset(testset, indices)

    testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
                                            shuffle=False,␣
↪num_workers=num_workers)

    classes = ('plane', 'car', 'bird', 'cat',
            'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
    return trainloader, testloader, classes
```

[3]:
```
trainloader, testloader, _ = prepare_data(batch_size=40)
```

```
/Users/valefeve/Projects/interpretable-transformers/venv/lib/python3.11/site-
packages/torchvision/datasets/cifar.py:83: VisibleDeprecationWarning: dtype():
align should be passed as Python or NumPy boolean but got `align=0`. Did you
mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
```

```
  entry = pickle.load(f, encoding="latin1")
```

[4]:
```
images = load_images_from_indices(testloader.dataset, [0, 1, 2, 3])
images
```

```
/Users/valefeve/Projects/interpretable-transformers/venv/lib/python3.11/site-
packages/torchvision/transforms/functional.py:154: UserWarning: The given NumPy
array is not writable, and PyTorch does not support non-writable tensors. This
means writing to this tensor will result in undefined behavior. You may want to
copy the array to protect its data or make it writable before converting it to a
tensor. This type of warning will be suppressed for the rest of this program.
(Triggered internally at /Users/runner/work/pytorch/pytorch/pytorch/torch/csrc/u
tils/tensor_numpy.cpp:219.)
  img = torch.from_numpy(pic.transpose((2, 0, 1))).contiguous()
```

[4]:
```
tensor([[[[ 0.2392,  0.2471,  0.2941,  …,  0.0745, -0.0118, -0.0902],
          [ 0.1922,  0.1843,  0.2471,  …,  0.0667, -0.0196, -0.0667],
          [ 0.1843,  0.1843,  0.2392,  …,  0.0902,  0.0196, -0.0588],
          …,
          [-0.4667, -0.6706, -0.7569,  …, -0.7020, -0.8980, -0.6863],
          [-0.5216, -0.6157, -0.7255,  …, -0.7961, -0.7725, -0.8431],
          [-0.5765, -0.5608, -0.6471,  …, -0.8118, -0.7333, -0.8353]],

         [[-0.1216, -0.1294, -0.0902,  …, -0.2549, -0.2863, -0.3333],
          [-0.1216, -0.1373, -0.1059,  …, -0.2549, -0.2863, -0.3098],
          [-0.1373, -0.1451, -0.1294,  …, -0.2314, -0.2549, -0.3020],
          …,
          [-0.0275, -0.2157, -0.3098,  …, -0.2392, -0.4980, -0.3333],
          [-0.0902, -0.2000, -0.3333,  …, -0.3569, -0.3569, -0.4980],
          [-0.1608, -0.1765, -0.3020,  …, -0.3961, -0.3412, -0.4745]],

         [[-0.6157, -0.6314, -0.6000,  …, -0.7176, -0.7176, -0.7412],
          [-0.6000, -0.6863, -0.6471,  …, -0.7569, -0.7490, -0.7333],
          [-0.6314, -0.7412, -0.7176,  …, -0.7333, -0.7333, -0.7412],
          …,
          [ 0.3882,  0.1608,  0.0745,  …,  0.1451, -0.1529, -0.0039],
          [ 0.3176,  0.1608,  0.0353,  …,  0.0196, -0.0118, -0.1608],
          [ 0.2549,  0.1686,  0.0353,  …, -0.0275,  0.0118, -0.1373]]],


        [[[ 0.8431,  0.8118,  0.8196,  …,  0.8275,  0.8275,  0.8196],
          [ 0.8667,  0.8431,  0.8431,  …,  0.8510,  0.8510,  0.8431],
          [ 0.8588,  0.8353,  0.8353,  …,  0.8431,  0.8431,  0.8353],
          …,
          [-0.3176, -0.6627, -0.8510,  …,  0.3255,  0.4275,  0.4745],
          [-0.3569, -0.6392, -0.7176,  …,  0.3647,  0.4510,  0.4667],
          [-0.3333, -0.5137, -0.5451,  …,  0.3176,  0.4118,  0.4588]],
```

```
 [[ 0.8431,  0.8118,  0.8196,  …,  0.8275,  0.8275,  0.8196],
  [ 0.8667,  0.8431,  0.8431,  …,  0.8510,  0.8510,  0.8431],
  [ 0.8588,  0.8353,  0.8353,  …,  0.8431,  0.8431,  0.8353],
  …,
  [-0.2235, -0.6000, -0.8196,  …,  0.4431,  0.5451,  0.5843],
  [-0.2471, -0.5529, -0.6549,  …,  0.4824,  0.5686,  0.5843],
  [-0.2078, -0.4118, -0.4745,  …,  0.4353,  0.5294,  0.5686]],

 [[ 0.8431,  0.8118,  0.8196,  …,  0.8275,  0.8275,  0.8196],
  [ 0.8667,  0.8431,  0.8431,  …,  0.8510,  0.8510,  0.8431],
  [ 0.8588,  0.8353,  0.8353,  …,  0.8431,  0.8431,  0.8353],
  …,
  [-0.3020, -0.7098, -0.9137,  …,  0.4039,  0.5137,  0.5765],
  [-0.3569, -0.7176, -0.8275,  …,  0.4353,  0.5373,  0.5686],
  [-0.3490, -0.6235, -0.7020,  …,  0.3961,  0.4980,  0.5608]]],


[[[ 0.2392,  0.2392,  0.0902,  …,  0.7882,  0.8588,  0.8667],
  [ 0.3333,  0.3490,  0.1843,  …,  0.8196,  0.9294,  0.9294],
  [ 0.3647,  0.3804,  0.2314,  …,  0.8039,  0.9608,  0.9216],
  …,
  [-0.7569, -0.7647, -0.7961,  …, -0.7098, -0.9294, -0.9686],
  [-0.8196, -0.7882, -0.8039,  …, -0.8510, -0.9686, -0.9608],
  [-0.7804, -0.7647, -0.7490,  …, -0.9608, -0.9686, -0.9451]],

 [[ 0.4902,  0.4667,  0.3020,  …,  0.8118,  0.8745,  0.8902],
  [ 0.5686,  0.5608,  0.3804,  …,  0.8196,  0.9294,  0.9373],
  [ 0.5765,  0.5686,  0.4039,  …,  0.7961,  0.9529,  0.9137],
  …,
  [-0.6863, -0.6941, -0.7255,  …, -0.6863, -0.8980, -0.9451],
  [-0.7333, -0.7020, -0.7176,  …, -0.8431, -0.9529, -0.9451],
  [-0.6784, -0.6627, -0.6471,  …, -0.9529, -0.9608, -0.9373]],

 [[ 0.7412,  0.7098,  0.5216,  …,  0.8353,  0.9059,  0.9294],
  [ 0.7961,  0.7725,  0.5765,  …,  0.8510,  0.9608,  0.9686],
  [ 0.7647,  0.7412,  0.5608,  …,  0.8196,  0.9686,  0.9373],
  …,
  [-0.6471, -0.6549, -0.6863,  …, -0.6392, -0.8902, -0.9608],
  [-0.6941, -0.6627, -0.6784,  …, -0.8118, -0.9765, -0.9765],
  [-0.6314, -0.6078, -0.5922,  …, -0.9373, -0.9765, -0.9451]]],


[[[ 0.2157,  0.3098,  0.3804,  …,  0.5765,  0.5843,  0.5059],
  [ 0.2000,  0.2784,  0.3412,  …,  0.5843,  0.5922,  0.4824],
  [ 0.2157,  0.2549,  0.3176,  …,  0.6157,  0.6000,  0.4824],
  …,
  [-0.3020, -0.5529, -0.5216,  …, -0.3020, -0.5373, -0.4745],
```

```
          [-0.3020, -0.5294, -0.5059,  …, -0.5529, -0.5216, -0.4118],
          [-0.2784, -0.5294, -0.5216,  …, -0.5294, -0.4980, -0.4275]],

         [[ 0.2235,  0.3804,  0.4039,  …,  0.5373,  0.5843,  0.4353],
          [ 0.2157,  0.4039,  0.4431,  …,  0.6314,  0.6863,  0.4902],
          [ 0.2078,  0.3961,  0.4510,  …,  0.6314,  0.6863,  0.4980],
          …,
          [-0.3804, -0.5843, -0.5216,  …, -0.0667, -0.3804, -0.4196],
          [-0.3569, -0.5451, -0.4824,  …, -0.4824, -0.4980, -0.4588],
          [-0.3882, -0.5922, -0.5451,  …, -0.5059, -0.4902, -0.4667]],

         [[ 0.1686,  0.4667,  0.5137,  …,  0.6392,  0.6627,  0.3412],
          [ 0.2314,  0.6000,  0.6863,  …,  0.7412,  0.7804,  0.4353],
          [ 0.2000,  0.5765,  0.6706,  …,  0.6863,  0.7569,  0.4431],
          …,
          [-0.5137, -0.5922, -0.5373,  …, -0.2078, -0.5137, -0.6078],
          [-0.5137, -0.5843, -0.5451,  …, -0.5451, -0.5765, -0.6000],
          [-0.4980, -0.6000, -0.6000,  …, -0.6078, -0.5922, -0.6078]]]],
       device='mps:0')
```

[5]:
```python
config = {
    "patch_size": 4,  # Input image size: 32x32 -> 8x8 patches
    "hidden_size": 48,
    "num_hidden_layers": 4,
    "num_attention_heads": 4,
    "intermediate_size": 4 * 48, # 4 * hidden_size
    "hidden_dropout_prob": 0.0,
    "attention_probs_dropout_prob": 0.0,
    "initializer_range": 0.02,
    "image_size": 32,
    "num_classes": 10, # num_classes of CIFAR10
    "num_channels": 3,
    "qkv_bias": True,
    "use_faster_attention": False,
}
```

[6]:
```python
model = ViTForClassfication(config=config)
```

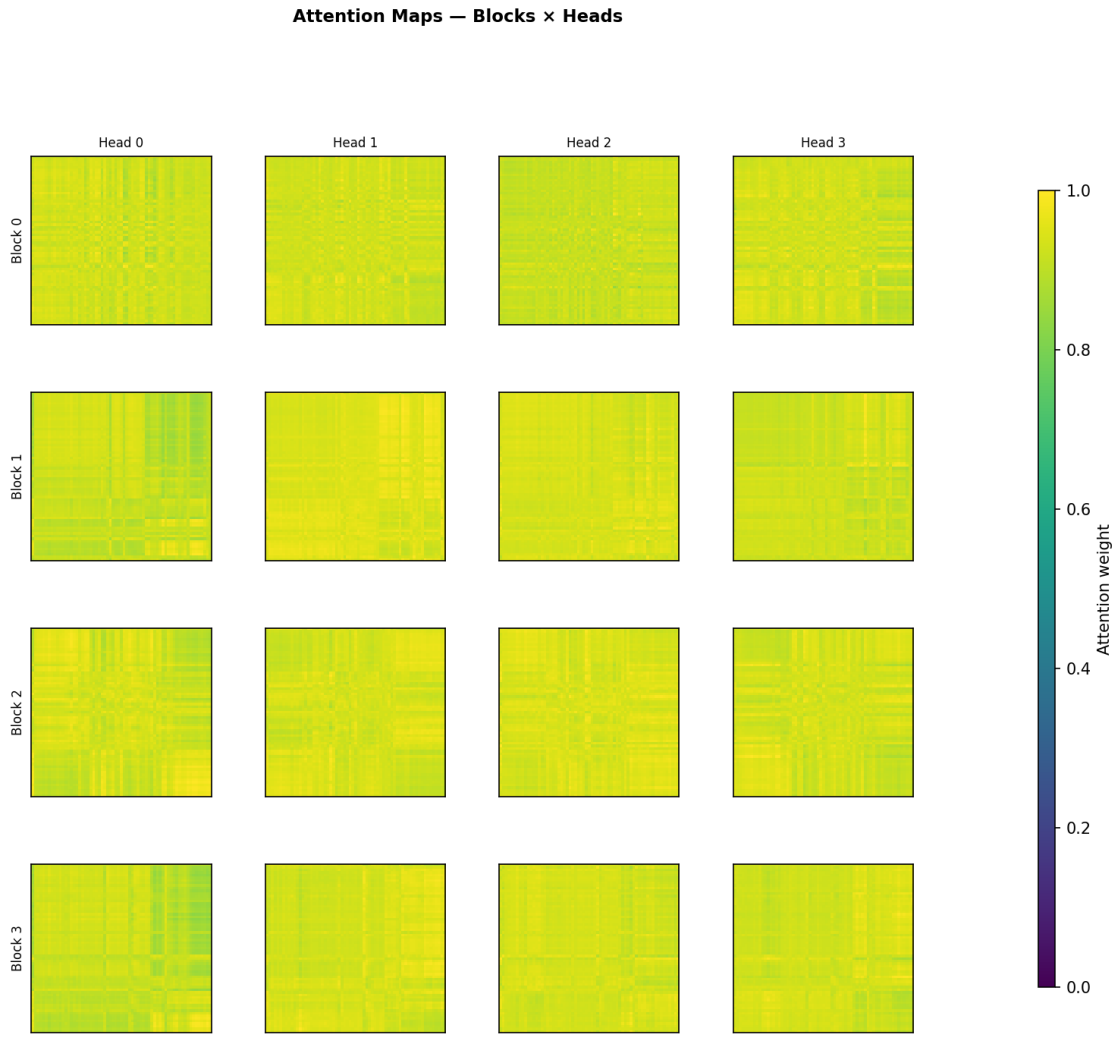
[7]:
```python
input_images = images.to(next(model.parameters()).device)
logits, attention_maps = model(input_images, output_attentions=True)
```

[10]:
```python
attention_maps[0].shape
```

[10]:
```
torch.Size([4, 4, 65, 65])
```

[13]:
```python
images_of_attention = visualize_attention(attention_maps[0])
images_of_attention
```

[13]:



[14]:
```
for image in images_of_attention:
    plt.imshow(image)
    plt.axis('off')
    plt.show()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[14], line 1
----> 1 for image in images_of_attention:
      2     plt.imshow(image)
      3     plt.axis(     )

TypeError: 'Image' object is not iterable
```

[15]:
```
images
```

[15]:
```
tensor([[[[ 0.2392,  0.2471,  0.2941,  …,  0.0745, -0.0118, -0.0902],
          [ 0.1922,  0.1843,  0.2471,  …,  0.0667, -0.0196, -0.0667],
          [ 0.1843,  0.1843,  0.2392,  …,  0.0902,  0.0196, -0.0588],
          …,
          [-0.4667, -0.6706, -0.7569,  …, -0.7020, -0.8980, -0.6863],
          [-0.5216, -0.6157, -0.7255,  …, -0.7961, -0.7725, -0.8431],
          [-0.5765, -0.5608, -0.6471,  …, -0.8118, -0.7333, -0.8353]],

         [[-0.1216, -0.1294, -0.0902,  …, -0.2549, -0.2863, -0.3333],
          [-0.1216, -0.1373, -0.1059,  …, -0.2549, -0.2863, -0.3098],
          [-0.1373, -0.1451, -0.1294,  …, -0.2314, -0.2549, -0.3020],
          …,
          [-0.0275, -0.2157, -0.3098,  …, -0.2392, -0.4980, -0.3333],
          [-0.0902, -0.2000, -0.3333,  …, -0.3569, -0.3569, -0.4980],
          [-0.1608, -0.1765, -0.3020,  …, -0.3961, -0.3412, -0.4745]],

         [[-0.6157, -0.6314, -0.6000,  …, -0.7176, -0.7176, -0.7412],
          [-0.6000, -0.6863, -0.6471,  …, -0.7569, -0.7490, -0.7333],
          [-0.6314, -0.7412, -0.7176,  …, -0.7333, -0.7333, -0.7412],
          …,
          [ 0.3882,  0.1608,  0.0745,  …,  0.1451, -0.1529, -0.0039],
          [ 0.3176,  0.1608,  0.0353,  …,  0.0196, -0.0118, -0.1608],
          [ 0.2549,  0.1686,  0.0353,  …, -0.0275,  0.0118, -0.1373]]],


        [[[ 0.8431,  0.8118,  0.8196,  …,  0.8275,  0.8275,  0.8196],
          [ 0.8667,  0.8431,  0.8431,  …,  0.8510,  0.8510,  0.8431],
          [ 0.8588,  0.8353,  0.8353,  …,  0.8431,  0.8431,  0.8353],
          …,
          [-0.3176, -0.6627, -0.8510,  …,  0.3255,  0.4275,  0.4745],
          [-0.3569, -0.6392, -0.7176,  …,  0.3647,  0.4510,  0.4667],
          [-0.3333, -0.5137, -0.5451,  …,  0.3176,  0.4118,  0.4588]],

         [[ 0.8431,  0.8118,  0.8196,  …,  0.8275,  0.8275,  0.8196],
          [ 0.8667,  0.8431,  0.8431,  …,  0.8510,  0.8510,  0.8431],
          [ 0.8588,  0.8353,  0.8353,  …,  0.8431,  0.8431,  0.8353],
          …,
          [-0.2235, -0.6000, -0.8196,  …,  0.4431,  0.5451,  0.5843],
          [-0.2471, -0.5529, -0.6549,  …,  0.4824,  0.5686,  0.5843],
          [-0.2078, -0.4118, -0.4745,  …,  0.4353,  0.5294,  0.5686]],

         [[ 0.8431,  0.8118,  0.8196,  …,  0.8275,  0.8275,  0.8196],
          [ 0.8667,  0.8431,  0.8431,  …,  0.8510,  0.8510,  0.8431],
          [ 0.8588,  0.8353,  0.8353,  …,  0.8431,  0.8431,  0.8353],
          …,
```

```
  [-0.3020, -0.7098, -0.9137,  …,  0.4039,  0.5137,  0.5765],
  [-0.3569, -0.7176, -0.8275,  …,  0.4353,  0.5373,  0.5686],
  [-0.3490, -0.6235, -0.7020,  …,  0.3961,  0.4980,  0.5608]]],


[[[ 0.2392,  0.2392,  0.0902,  …,  0.7882,  0.8588,  0.8667],
  [ 0.3333,  0.3490,  0.1843,  …,  0.8196,  0.9294,  0.9294],
  [ 0.3647,  0.3804,  0.2314,  …,  0.8039,  0.9608,  0.9216],
  …,
  [-0.7569, -0.7647, -0.7961,  …, -0.7098, -0.9294, -0.9686],
  [-0.8196, -0.7882, -0.8039,  …, -0.8510, -0.9686, -0.9608],
  [-0.7804, -0.7647, -0.7490,  …, -0.9608, -0.9686, -0.9451]],

 [[ 0.4902,  0.4667,  0.3020,  …,  0.8118,  0.8745,  0.8902],
  [ 0.5686,  0.5608,  0.3804,  …,  0.8196,  0.9294,  0.9373],
  [ 0.5765,  0.5686,  0.4039,  …,  0.7961,  0.9529,  0.9137],
  …,
  [-0.6863, -0.6941, -0.7255,  …, -0.6863, -0.8980, -0.9451],
  [-0.7333, -0.7020, -0.7176,  …, -0.8431, -0.9529, -0.9451],
  [-0.6784, -0.6627, -0.6471,  …, -0.9529, -0.9608, -0.9373]],

 [[ 0.7412,  0.7098,  0.5216,  …,  0.8353,  0.9059,  0.9294],
  [ 0.7961,  0.7725,  0.5765,  …,  0.8510,  0.9608,  0.9686],
  [ 0.7647,  0.7412,  0.5608,  …,  0.8196,  0.9686,  0.9373],
  …,
  [-0.6471, -0.6549, -0.6863,  …, -0.6392, -0.8902, -0.9608],
  [-0.6941, -0.6627, -0.6784,  …, -0.8118, -0.9765, -0.9765],
  [-0.6314, -0.6078, -0.5922,  …, -0.9373, -0.9765, -0.9451]]],


[[[ 0.2157,  0.3098,  0.3804,  …,  0.5765,  0.5843,  0.5059],
  [ 0.2000,  0.2784,  0.3412,  …,  0.5843,  0.5922,  0.4824],
  [ 0.2157,  0.2549,  0.3176,  …,  0.6157,  0.6000,  0.4824],
  …,
  [-0.3020, -0.5529, -0.5216,  …, -0.3020, -0.5373, -0.4745],
  [-0.3020, -0.5294, -0.5059,  …, -0.5529, -0.5216, -0.4118],
  [-0.2784, -0.5294, -0.5216,  …, -0.5294, -0.4980, -0.4275]],

 [[ 0.2235,  0.3804,  0.4039,  …,  0.5373,  0.5843,  0.4353],
  [ 0.2157,  0.4039,  0.4431,  …,  0.6314,  0.6863,  0.4902],
  [ 0.2078,  0.3961,  0.4510,  …,  0.6314,  0.6863,  0.4980],
  …,
  [-0.3804, -0.5843, -0.5216,  …, -0.0667, -0.3804, -0.4196],
  [-0.3569, -0.5451, -0.4824,  …, -0.4824, -0.4980, -0.4588],
  [-0.3882, -0.5922, -0.5451,  …, -0.5059, -0.4902, -0.4667]],

 [[ 0.1686,  0.4667,  0.5137,  …,  0.6392,  0.6627,  0.3412],
```

```
            [ 0.2314,  0.6000,  0.6863,  …,  0.7412,  0.7804,  0.4353],
            [ 0.2000,  0.5765,  0.6706,  …,  0.6863,  0.7569,  0.4431],
            …,
            [-0.5137, -0.5922, -0.5373,  …, -0.2078, -0.5137, -0.6078],
            [-0.5137, -0.5843, -0.5451,  …, -0.5451, -0.5765, -0.6000],
            [-0.4980, -0.6000, -0.6000,  …, -0.6078, -0.5922, -0.6078]]]],
        device='mps:0')
```

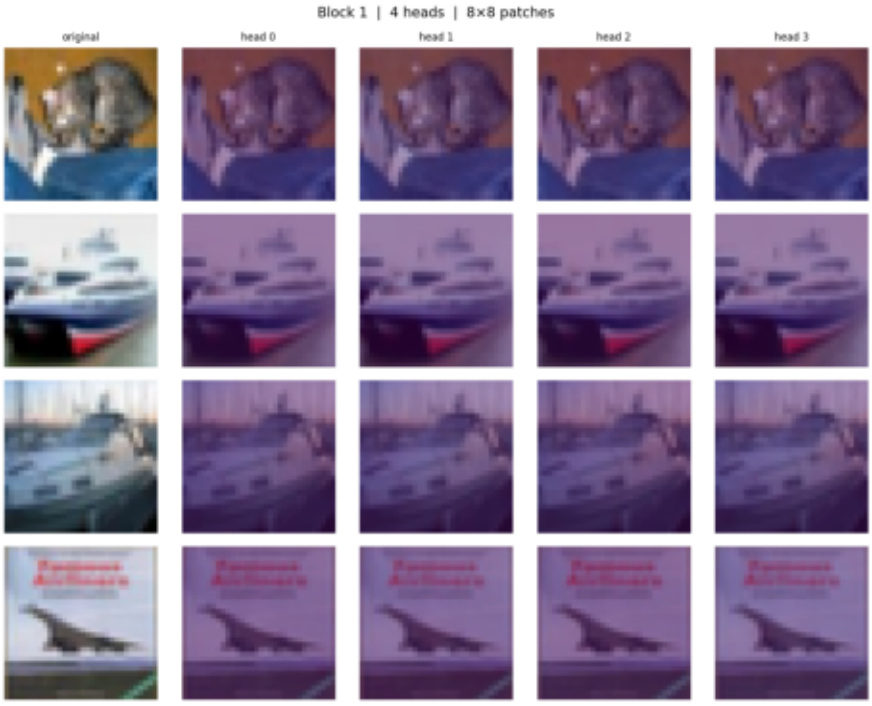
[16]:
```
print(attention_maps[0].shape)  # (batch_size, num_heads, num_patches + 1, num_patches + 1)
print(images.shape)  # Attention map for the first head of the first layer
```

```
torch.Size([4, 4, 65, 65])
torch.Size([4, 3, 32, 32])
```

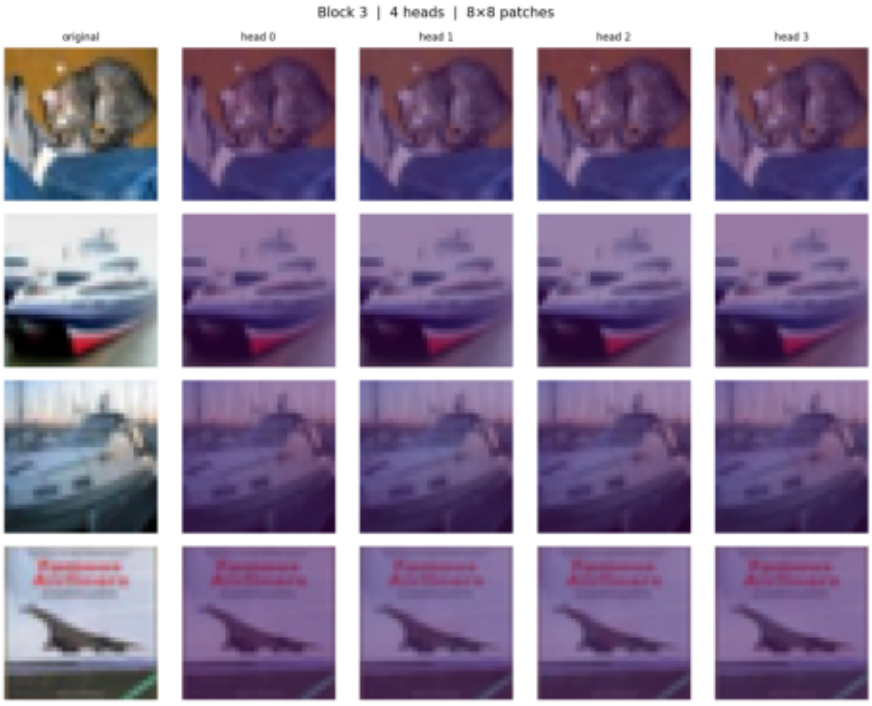
[17]:
```
attention_of_images = visualize_attention_on_images(attention_maps, input_images, alpha=0.5)
```

[18]:
```
for image in attention_of_images:
    plt.imshow(image)
    plt.axis('off')
    plt.show()
```




Block 1 | 4 heads | 8×8 patches
original
head 0
head 1
head 2
head 3


Block 2 | 4 heads | 8×8 patches
original
head 0
head 1
head 2
head 3


Block 3 | 4 heads | 8×8 patches
original
head 0
head 1
head 2
head 3

[ ]: